

TRƯỜNG ĐẠI HỌC THỦ DẦU MỘT
VIỆN CÔNG NGHỆ SỐ

BÁO CÁO ĐỒ ÁN

HỌC PHẦN: CHUYÊN ĐỀ INTERNET OF THINGS

NỀN TẢNG QUẢN LÝ NHIỀU THIẾT BỊ IoT

Sinh viên thực hiện:

STT	Họ và tên	MSSV	Lớp
1	[Bổ sung]	[Bổ sung]	[Bổ sung]
2	[Bổ sung]	[Bổ sung]	[Bổ sung]
3	[Bổ sung]	[Bổ sung]	[Bổ sung]

Lớp học phần:	[Bổ sung lớp]
Nhóm:	[Bổ sung tên nhóm]
Giảng viên hướng dẫn:	[Bổ sung họ tên giảng viên]

TP. HỒ CHÍ MINH, NĂM 2026

BẢNG PHÂN CÔNG THÀNH VIÊN

Điền thông tin thành viên và phần việc theo hồ sơ nhóm trước khi nộp.

STT	Họ và tên	MSSV	Nhiệm vụ thực hiện	Sản phẩm/kết quả	Mức độ đóng góp
1	[Bổ sung]	[Bổ sung]	[Xác nhận]	[Xác nhận]	[Xác nhận]
2	[Bổ sung]	[Bổ sung]	[Xác nhận]	[Xác nhận]	[Xác nhận]
3	[Bổ sung]	[Bổ sung]	[Xác nhận]	[Xác nhận]	[Xác nhận]

MỤC LỤC

MỞ ĐẦU

CHƯƠNG 1. TỔNG QUAN HỆ THỐNG VÀ CÔNG NGHỆ LIÊN QUAN	2
1.1. Tổng quan bài toán, mục tiêu và phạm vi đề tài	2
1.2. Mô hình tổng quan của nền tảng quản lý nhiều thiết bị IoT	4
1.3. Công nghệ và giao thức sử dụng	6
CHƯƠNG 2. THIẾT KẾ PHẦN CỨNG, PHẦN MỀM VÀ MÔ HÌNH HOẠT ĐỘNG	8
2.1. Phân tích yêu cầu và kiến trúc thiết kế	8
2.2. Thiết kế quản lý thiết bị và dữ liệu	10
2.3. Thiết kế giao tiếp, giám sát và điều khiển thiết bị	14
2.4. Thiết kế giao diện và luồng sử dụng hệ thống	17
CHƯƠNG 3. TRIỂN KHAI, THỬ NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG	20
3.1. Môi trường và quá trình triển khai	20
3.2. Kết quả xây dựng hệ thống	21
3.3. Thử nghiệm quản lý nhiều thiết bị IoT	24
3.4. Đánh giá kết quả	27
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	29
TÀI LIỆU THAM KHẢO	

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Diễn giải
IoT	Internet of Things
MQTT	Message Queuing Telemetry Transport
API	Application Programming Interface

HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
RLS	Row-Level Security
SSE	Server-Sent Events
UID	Unique Identifier
OTA	Over-the-Air

DANH MỤC BẢNG

Bảng 1.1. Mục tiêu và tiêu chí đánh giá của đề tài	2
Bảng 1.2. Phạm vi tổ chức trong nền tảng	4
Bảng 1.3. Công nghệ và vai trò trong chuỗi quản lý thiết bị	6
Bảng 2.1. Yêu cầu chức năng trọng tâm	8
Bảng 2.2. Yêu cầu phi chức năng và ảnh hưởng thiết kế	9
Bảng 2.3. Trách nhiệm của các lớp và thành phần	10
Bảng 2.4. Trạng thái vòng đời và thao tác quản lý tương ứng	11
Bảng 2.5. Các thực thể dữ liệu chính	13
Bảng 2.6. Topic MQTT trong phạm vi báo cáo	14
Bảng 2.7. Thông tin ưu tiên trên từng màn hình	18
Bảng 3.1. Thành phần triển khai và vai trò vận hành	20
Bảng 3.2. Kết quả xây dựng nhìn từ hành trình người sử dụng	23
Bảng 3.3. Kết quả thử nghiệm và quan sát đã thực hiện	24
Bảng 3.4. Số bản ghi theo thiết bị trong snapshot database	26
Bảng 3.5. Đối chiếu mục tiêu với mức xác minh hiện tại	27
Bảng A.1. Vết liên kết giữa chức năng và mã nguồn	31

DANH MỤC HÌNH

Hình 1.1. Kiến trúc tổng thể của nền tảng	4
Hình 2.1. Ca sử dụng trọng tâm của hệ thống	8
Hình 2.2. Vòng đời của một thiết bị trong nền tảng	11
Hình 2.3. Quan hệ dữ liệu phục vụ quản lý thiết bị	13
Hình 2.4. Trình tự kết nối và thiết lập hiện diện	14
Hình 2.5. Luồng telemetry giữ nguyên định danh thiết bị	15
Hình 2.6. Chuỗi điều khiển và xác nhận lệnh	16
Hình 2.7. Hành trình người dùng quản lý một thiết bị	18
Hình 3.1. Bố trí các thành phần trong môi trường triển khai	20

Hình 3.2. Giao diện danh sách nhiều thiết bị	22
Hình 3.3. Luồng tạo và đăng ký thiết bị	22
Hình 3.4. Giao diện datastream và dữ liệu thiết bị	23
Hình 3.5. Giao diện trạng thái và điều khiển thiết bị	23
Hình 3.6. Phân bố số bản ghi telemetry trong ảnh chụp dữ liệu ngày 27/09/2026	26
Hình 3.7. Bảng chứng phiên thử nghiệm nhiều thiết bị	27

MỞ ĐẦU

Các hệ thống IoT thường bắt đầu từ một thiết bị đơn lẻ: một bo mạch đọc cảm biến, gửi dữ liệu và nhận một vài lệnh điều khiển. Khi số lượng thiết bị tăng, khó khăn không còn nằm ở việc đọc được một giá trị cảm biến mà chuyển sang bài toán quản lý. Mỗi thiết bị có định danh, thông tin kết nối, trạng thái, dữ liệu lịch sử và quyền truy cập riêng. Nếu các yếu tố đó được theo dõi bằng nhiều công cụ rời rạc, người vận hành khó biết thiết bị nào đang hoạt động, dữ liệu thuộc về đâu và lệnh đã đến đúng đích hay chưa.

Đề tài “Nền tảng quản lý nhiều thiết bị IoT” được thực hiện để tập trung các công việc trên vào một hệ thống thống nhất. Nền tảng duy trì danh mục thiết bị theo tenant và project; dùng UID làm định danh xuyên suốt từ cơ sở dữ liệu tới topic MQTT; tiếp nhận telemetry, status và heartbeat; cung cấp datastream theo virtual pin; đồng thời cho phép người dùng gửi lệnh tới một thiết bị cụ thể. Giao diện web phục vụ thao tác quản lý hằng ngày, ứng dụng Android cung cấp luồng xem thiết bị trên di động, còn firmware ESP32 và simulator đóng vai trò đầu cuối kết nối tới broker.

Mục tiêu của đề án là mô tả và hiện thực một đường đi đầy đủ cho dữ liệu thiết bị. Sau khi một Device được tạo, hệ thống cấp thông tin cần thiết để client MQTT kết nối. Bản tin đi lên phải được nhận diện theo UID trước khi lưu với device_id tương ứng. Khi người dùng mở trang chi tiết, API truy vấn đúng dữ liệu của thiết bị trong phạm vi tenant hoặc project. Ở chiều ngược lại, lệnh từ giao diện được kiểm tra quyền, phát vào topic chứa UID và chỉ được xem là đã thực thi khi có ACK hoặc dữ liệu phản hồi từ thiết bị.

Phạm vi báo cáo tập trung vào registry, provisioning, project, device group, datastream, telemetry, hiện diện online/offline, điều khiển, phân quyền và các giao diện trực tiếp phục vụ quản lý thiết bị. Các module không tham gia vào chuỗi nghiệp vụ này không được dùng để mở rộng thành tích của đề tài. Báo cáo cũng không công bố số liệu hiệu năng, độ trễ hoặc số thiết bị đồng thời khi chưa có phép đo có thể tái lập.

CHƯƠNG 1. TỔNG QUAN HỆ THỐNG VÀ CÔNG NGHỆ LIÊN QUAN

1.1. Tổng quan bài toán, mục tiêu và phạm vi đề tài

Bài toán phát sinh khi số lượng thiết bị tăng

Với một thiết bị, người phát triển có thể nhớ UID, địa chỉ broker và loại dữ liệu mà thiết bị gửi. Cách làm này không còn phù hợp khi một tenant có nhiều thiết bị ở các dự án khác nhau. Danh sách cần trả lời được thiết bị nào thuộc đơn vị nào, đang được gán vào project nào, đã xuất hiện lần cuối lúc nào và có kênh dữ liệu nào. Nếu chỉ nhìn vào topic hoặc một bảng telemetry chung, người dùng khó phát hiện bản ghi bị gán nhầm thiết bị và càng khó thao tác theo nhóm.

Dữ liệu của nhiều thiết bị còn khác nhau về nhịp gửi và kiểu giá trị. Một thiết bị có thể gửi nhiệt độ định kỳ, thiết bị khác báo trạng thái relay khi có thay đổi, còn một client mô phỏng có thể phát nhiều metric trong cùng payload. Nền tảng vì vậy phải tách cấu hình kênh khỏi bản ghi đo. Datastream mô tả ý nghĩa của kênh trong project; telemetry ghi lại giá trị phát sinh theo thời gian và theo device_id. Việc tách này giúp cùng một cấu trúc dashboard phục vụ nhiều thiết bị mà không làm mất nguồn gốc dữ liệu.

Trạng thái kết nối cũng không thể suy ra từ một cò online đơn giản. Thiết bị có thể mất mạng mà không kịp gửi thông điệp offline. Hệ thống cần theo dõi heartbeat, status và thời điểm last_seen_at, sau đó áp dụng ngưỡng offline riêng cho thiết bị. Người vận hành phải thấy thiết bị A chuyển offline mà không ảnh hưởng nhân trạng thái của B. Đây là yêu cầu quan trọng đối với một nền tảng nhiều thiết bị vì trạng thái sai có thể dẫn đến thao tác lệnh trên một đầu cuối không còn kết nối.

Điều khiển đặt ra một yêu cầu khác: phản hồi HTTP chỉ cho biết backend đã tiếp nhận yêu cầu. Nó không chứng minh thiết bị đã nhận hoặc thực thi lệnh. Hệ thống cần gán command_id, phát đúng topic của UID và chờ event ACK hoặc telemetry phản hồi. Nhờ đó, giao diện có thể phân biệt các trạng thái đã gửi, đang chờ, đã xác nhận hoặc thất bại thay vì hiển thị thành công ngay sau HTTP 2xx.

Mục tiêu và tiêu chí của đề án

Bảng 1.1. Mục tiêu và tiêu chí đánh giá của đề tài

Mục tiêu	Kết quả cần có	Cách nhận biết
Quản lý tập trung nhiều thiết bị	Danh sách, tìm kiếm, lọc và chi tiết theo UID	Không tạo trùng UID; mở đúng thiết bị
Tổ chức theo phạm vi	Tenant, project và nhóm thiết bị	Danh sách toàn tenant khác workspace project
Tiếp nhận dữ liệu	Telemetry gán device_id và thời gian	Truy vấn A không trả dữ liệu của B

Theo dõi kết nối	Status, heartbeat, last_seen và timeout	Mỗi thiết bị có trạng thái độc lập
Điều khiển từ xa	Command theo UID và ACK theo command_id	Không đồng nhất HTTP 2xx với thực thi
Kiểm soát truy cập	JWT, quyền ứng dụng và RLS	Người dùng chỉ thấy tài nguyên được phép

Mục tiêu tổng quát là tạo một nền tảng mà người dùng có thể quan sát và thao tác với nhiều Device từ cùng một giao diện nhưng hệ thống vẫn giữ được danh tính riêng của từng đầu cuối. Bảng 1.1 chuyển mục tiêu đó thành các tiêu chí có thể kiểm tra. Tên “nhiều thiết bị” không được chứng minh bằng việc giao diện lập qua một mảng; nó cần sự nhất quán giữa UID, dữ liệu, trạng thái, lệnh và quyền truy cập.

Phạm vi chức năng và đối tượng sử dụng

Đối tượng chính là người dùng thuộc tenant. Tenant owner quản lý phạm vi tổ chức; tenant engineer thực hiện các thao tác kỹ thuật như đăng ký thiết bị, cấu hình datastream và điều khiển; viewer sử dụng các màn hình đọc. Admin quản lý ở cấp nền tảng, còn platform engineer có luồng hỗ trợ chỉ đọc theo các quyền platform. Thiết bị là một tác nhân độc lập: nó không dùng phiên đăng nhập web mà kết nối MQTT bằng danh tính và credential gắn với UID.

Phạm vi kỹ thuật gồm Device Registry, provisioning, liên kết tenant/project, device group, datastream V0–V255, telemetry, trạng thái kết nối, command, web, Android, firmware ESP32, simulator, API và broker MQTT. Báo cáo chỉ phân tích các thành phần tham gia trực tiếp vào vòng đời thiết bị. Việc triển khai phần cứng thật, khả năng chịu tải và kiểm thử xuyên tenant chỉ được coi là hoàn tất khi có ảnh, log hoặc kết quả chạy trên môi trường an toàn.

Các tình huống sai lệch mà thiết kế phải ngăn chặn

Bài toán nhiều thiết bị dễ phát sinh các lỗi không thấy khi chỉ chạy một client. Trường hợp thứ nhất là trùng hoặc dùng sai UID: hai client có thể cùng gửi vào một namespace, khiến lịch sử không còn phân biệt được nguồn. Trường hợp thứ hai là lọc theo project nhưng bỏ qua tenant, làm một UID ngoài phạm vi xuất hiện trong kết quả. Trường hợp thứ ba là lệnh gửi theo nhóm chỉ lưu một trạng thái chung, che mất một target thất bại. Trường hợp thứ tư là giao diện giữ trạng thái online cũ sau khi SSE ngắt. Các tình huống này được dùng làm câu hỏi thiết kế ở Chương 2 và tiêu chí thử nghiệm ở Chương 3.

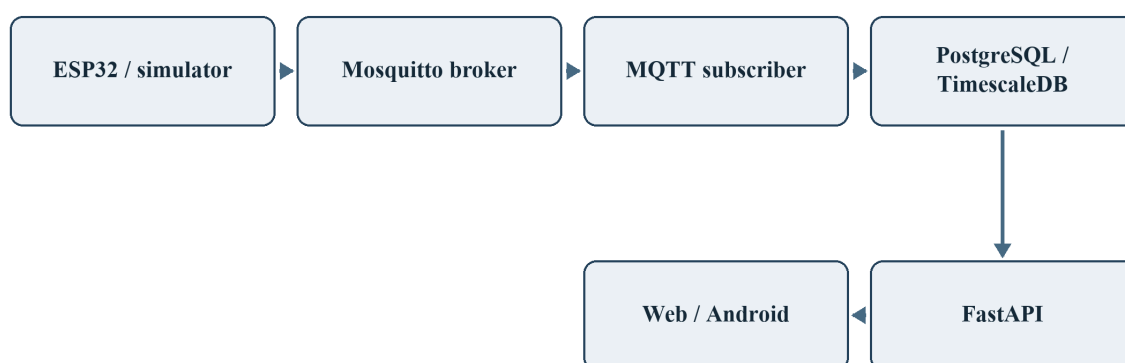
Một rủi ro khác nằm ở sự khác nhau giữa cấu hình và dữ liệu phát sinh. Datastream có thể được đổi alias hoặc direction trong khi telemetry cũ vẫn còn. Nếu giao diện chỉ đọc cấu hình hiện tại để giải thích toàn bộ lịch sử, người dùng có thể hiểu sai ý nghĩa của số đo cũ. Tương tự, việc chuyển Device sang project mới không được phép biến dữ liệu cũ thành dữ liệu của thiết bị khác. Thiết kế cần giữ khóa Device và timestamp làm mốc ổn định, còn cấu hình project/datastream là ngữ cảnh trình bày.

Bảng 1.2. Phạm vi tổ chức trong nền tảng

Khái niệm	Câu hỏi được trả lời	Ví dụ sử dụng
Tenant	Tài nguyên thuộc đơn vị nào?	Giới hạn người dùng, Device và project
Project	Thiết bị đang phục vụ bài toán/khu vực nào?	Workspace và datastream riêng
Device group	Những thiết bị nào cần được thao tác như một tập?	Lọc hoặc tạo dispatch nhiều target
Device	Đầu cuối nào tạo dữ liệu/nhận lệnh?	UID, trạng thái, telemetry và command
Datastream	Kênh Vn có ý nghĩa gì trong project?	Alias, kiểu và chiều dữ liệu

Bảng 1.2 cho thấy các lớp tổ chức không thay thế nhau. Tenant là ranh giới sở hữu, project là ngữ cảnh vận hành, group là tập hợp linh hoạt, Device là đầu cuối và datastream là mô tả kênh. Việc giữ đúng vai trò này giúp nền tảng mở rộng số thiết bị mà không phải nhân bản dữ liệu hoặc dùng một trường duy nhất cho nhiều mục đích.

1.2. Mô hình tổng quan của nền tảng quản lý nhiều thiết bị IoT

**Hình 1.1. Kiến trúc tổng thể của nền tảng**

Nguồn: Nhóm tác giả xây dựng từ mã nguồn.

Hình 1.1 mô tả sáu khối chính. ESP32 hoặc simulator tạo dữ liệu và duy trì kết nối. Mosquitto nhận bản tin publish/subscribe và áp dụng xác thực cùng ACL. MQTT subscriber của backend phân loại topic, lấy UID, kiểm tra payload rồi cập nhật Device hoặc tạo Telemetry. PostgreSQL lưu metadata; TimescaleDB hỗ trợ phân dữ liệu theo thời gian. FastAPI cung cấp API cho web và mobile, đồng thời MQTT publisher tạo chiều điều khiển từ nền tảng xuống thiết bị.

Chiều dữ liệu từ thiết bị lên giao diện

Khi một thiết bị gửi telemetry, UID không được lấy từ một trường tùy ý trên giao diện mà nằm trong topic. Subscriber dùng grammar topic đã thống nhất để tách

UID, tìm Device tương ứng và chỉ sau đó mới ghi dữ liệu với device_id. Cách này duy trì một đường truy vết rõ: topic cho biết client nào gửi, khóa ngoại cho biết bản ghi thuộc Device nào, còn tenant_id/project_id của Device quyết định người dùng nào được phép truy vấn.

Một yêu cầu đọc dữ liệu từ web đi theo chiều khác. Trình duyệt gọi API với phiên đã xác thực; backend lấy tenant từ danh tính hiện tại, kiểm tra project nếu có rồi truy vấn Device hoặc Telemetry. Kết quả trả về chứa UID, trạng thái và giá trị cần hiển thị. Giao diện toàn tenant dùng tập dữ liệu rộng để tìm thiết bị chưa gán; workspace chỉ trình bày thiết bị của project đang mở. Hai góc nhìn dùng chung bản ghi Device nhưng phục vụ hai nhiệm vụ vận hành khác nhau.

Chiều lệnh từ người dùng xuống thiết bị

Người dùng chọn một thiết bị và thao tác lệnh trên web hoặc mobile. API kiểm tra quyền thao tác và quyền sở hữu tài nguyên trước khi tạo yêu cầu phát. MQTT publisher chọn topic commands của UID hoặc topic virtual channel. Thiết bị đăng ký topic tương ứng, xử lý payload rồi gửi event ACK hoặc giá trị telemetry phản hồi. command_id giúp backend ghép phản hồi với yêu cầu ban đầu, đặc biệt khi nhiều thiết bị cùng nhận lệnh gần nhau.

Trong kịch bản dữ liệu và điều khiển, điểm cần giữ xuyên suốt là UID của thiết bị. Nếu bản tin của thiết bị A được lưu cho Device B hoặc publisher phát sang topic B, nền tảng đã vi phạm mục tiêu cốt lõi. Khi mở rộng sang nhiều thiết bị, cùng một chuỗi được thực hiện song song nhưng mỗi chuỗi vẫn có định danh và trạng thái riêng.

Vai trò của backend và cơ sở dữ liệu

Backend không chỉ chuyển tiếp bản tin. Nó là nơi ghép hai thế giới giao tiếp: HTTP dành cho người dùng và MQTT dành cho thiết bị. Ở phía HTTP, backend xác thực người dùng, áp dụng role, tenant và project. Ở phía MQTT, backend phân tích topic, kiểm tra UID, chuẩn hóa payload và cập nhật trạng thái. Các bounded context giữ nghiệp vụ registry, project, telemetry, provisioning và command trong cùng một modular monolith để một transaction có thể duy trì quan hệ dữ liệu mà không cần đồng bộ nhiều dịch vụ độc lập.

Cơ sở dữ liệu lưu hai nhóm thông tin. Metadata gồm tenant, user, project, Device, datastream, group và command dispatch. Dữ liệu vận hành gồm telemetry cùng thời điểm phát sinh, trạng thái và lịch sử mục tiêu lệnh. PostgreSQL RLS nhận ngữ cảnh tenant như một lớp bảo vệ bổ sung cho guard ở API [4]. RLS không thay thế việc kiểm tra quyền ứng dụng; hai lớp phải cùng đúng mới hạn chế được truy vấn chéo tenant.

Biên của mỗi giao thức trong kiến trúc

HTTP phù hợp với các thao tác có request/response rõ như đăng nhập, lấy danh sách, tạo Device, sửa cấu hình và truy vấn lịch sử. MQTT phù hợp với kết nối dài và bản tin phát sinh không đồng bộ từ thiết bị. SSE nằm giữa hai phía: trình duyệt vẫn dùng HTTP nhưng server có thể đẩy sự kiện trạng thái theo một chiều. Việc dùng

đúng giao thức cho từng biên giúp tránh để firmware phụ thuộc vào phiên web hoặc để trình duyệt phải đăng ký trực tiếp vào broker bằng credential thiết bị.

Ranh giới cũng làm rõ nơi xử lý lỗi. HTTP trả status code và body theo request. MQTT dùng kết quả kết nối, QoS, event ACK và log broker. SSE cần logic reconnect ở client. Database báo lỗi transaction hoặc ràng buộc. Khi người vận hành thấy một trạng thái không đổi, quy trình chẩn đoán đi từ snapshot REST, kết nối SSE, last_seen trong DB, log subscriber rồi mới tới broker và thiết bị. Cách truy vết này thực tế hơn việc chỉ kiểm tra giao diện cuối cùng.

1.3. Công nghệ và giao thức sử dụng

Bảng 1.3. Công nghệ và vai trò trong chuỗi quản lý thiết bị

Nhóm	Công nghệ	Vai trò trong đề tài
Thiết bị	ESP32, ESP-IDF	Kết nối Wi-Fi/MQTT, gửi status, heartbeat và telemetry
Mô phỏng	Python simulator	Tạo client MQTT theo UID để thử luồng nhiều thiết bị
Truyền thông	MQTT, Mosquitto	Định tuyến bản tin theo topic và UID
Backend	Python, FastAPI, SQLAlchemy	API, quyền truy cập, nghiệp vụ và tích hợp MQTT
Lưu trữ	PostgreSQL, TimescaleDB	Metadata và chuỗi telemetry theo thời gian
Web	React, Vite, TypeScript	Danh sách, workspace, detail và điều khiển
Mobile	Android Java	Danh sách, chi tiết và thao tác thiết bị trên di động
Triển khai	Docker Compose	Khai báo và kết nối các dịch vụ cục bộ

MQTT và Mosquitto

Bảng 1.3 hệ thống hóa công nghệ và vai trò trong chuỗi quản lý thiết bị. MQTT được chọn cho chiều giao tiếp với thiết bị vì mô hình publish/subscribe tách client phát dữ liệu khỏi dịch vụ xử lý. Thiết bị chỉ cần biết broker và namespace topic của mình; subscriber có thể nhận dữ liệu của nhiều UID bằng wildcard. Chuẩn MQTT định nghĩa ba mức QoS và cơ chế phiên kết nối [1]. Trong hợp đồng của dự án, telemetry ưu tiên QoS 0 để giảm chi phí cho dữ liệu lặp, còn status và command dùng QoS 1 để có xác nhận chuyển giao ở mức giao thức. QoS không thay thế ACK nghiệp vụ: broker xác nhận nhận bản tin không đồng nghĩa relay đã đổi trạng thái.

Mosquitto đóng vai trò broker trung tâm. Cấu hình ACL có thể giới hạn username của thiết bị chỉ được publish/subscribe trên topic chứa UID của nó [6]. Đây là lớp chặn sớm trước backend. Nếu credential của A có thể publish vào

namespace B, subscriber vẫn cần kiểm tra nhưng ranh giới thiết bị tại broker đã bị phá vỡ. Vì vậy credential, topic và UID phải được quản lý như một bộ thống nhất.

ESP32, ESP-IDF và simulator

Firmware mẫu dùng ESP-MQTT của ESP-IDF để tạo client, đăng ký topic lệnh, publish trạng thái khi kết nối và heartbeat định kỳ. Thư viện cung cấp API publish, subscribe và các sự kiện kết nối cần thiết cho vòng đời MQTT [2]. Trong đề án, firmware thể hiện cách một thiết bị vật lý tham gia nền tảng; simulator dùng cùng hợp đồng topic/payload để thử nhiều client mà không cần nhiều board. Hai loại client chỉ được xem tương đương ở những chức năng cùng hỗ trợ, không suy diễn simulator chứng minh hành vi phần cứng.

FastAPI, PostgreSQL và TimescaleDB

FastAPI tổ chức API theo router và sinh OpenAPI từ các path operation [3]. Đặc điểm này phù hợp với đề án vì nhóm có thể đối chiếu endpoint runtime với luồng giao diện. SQLAlchemy ánh xạ Device, Telemetry, Datastream và các quan hệ; session đặt ngữ cảnh tenant trước truy vấn. PostgreSQL đảm nhiệm dữ liệu quan hệ và RLS. TimescaleDB mở rộng PostgreSQL cho dữ liệu theo thời gian; hypertable chia dữ liệu thành các chunk theo khoảng thời gian để truy vấn đúng vùng dữ liệu thay vì quét toàn bộ [5]. Báo cáo không khẳng định telemetry hiện đã được tối ưu đầy đủ nếu chưa đo schema và kế hoạch truy vấn trên dữ liệu lớn.

React, Android, SSE và Docker Compose

React phù hợp với giao diện có nhiều trạng thái vì danh sách, bộ lọc, hộp thoại và bảng có thể tách thành component [9]. Web tải snapshot ban đầu qua REST, sau đó nghe SSE để nhận thay đổi trạng thái. SSE tạo kênh một chiều từ server tới trình duyệt bằng EventSource [8], phù hợp cho cập nhật hiện diện mà không cần polling liên tục. Android dùng cùng API ở góc nhìn di động; mức hoàn thiện phải được đánh giá theo màn hình và ViewModel thực tế.

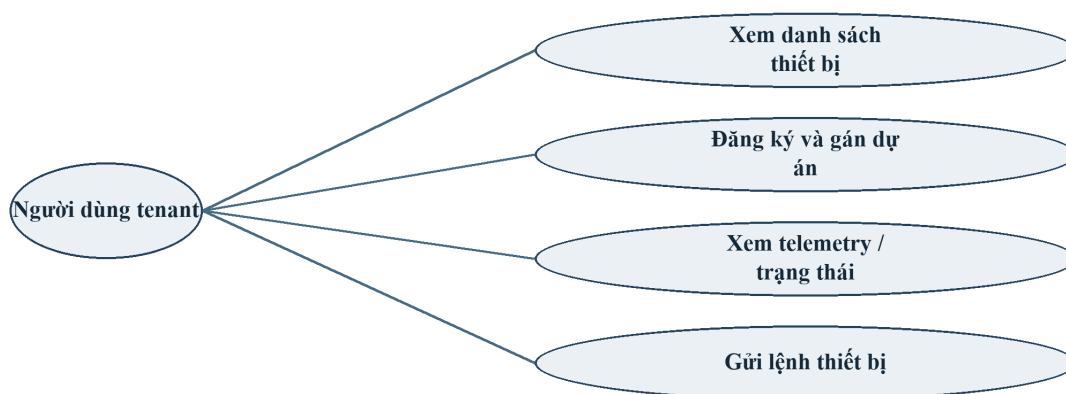
Docker Compose mô tả các service, network và volume của ứng dụng nhiều container trong một tệp cấu hình [7]. Trong môi trường phát triển, API, PostgreSQL/TimescaleDB, Mosquitto và các dịch vụ phụ trợ có thể khởi động nhất quán. Compose giúp tái lập kết nối giữa các thành phần nhưng không tự đảm bảo dữ liệu test được cách ly. Database dùng cho pytest vẫn phải được kiểm tra bằng connection target thực trước khi chạy.

CHƯƠNG 2. THIẾT KẾ PHẦN CỨNG, PHẦN MỀM VÀ MÔ HÌNH HOẠT ĐỘNG

2.1. Phân tích yêu cầu và kiến trúc thiết kế

Tác nhân và nhu cầu sử dụng

Tenant owner cần nhìn toàn bộ thiết bị của đơn vị, biết thiết bị nào chưa được gán project và quản lý thành viên có quyền thao tác. Tenant engineer cần đăng ký, cấu hình, theo dõi telemetry và phát lệnh. Viewer cần đọc danh sách và dữ liệu mà không làm thay đổi tài nguyên. Admin và platform engineer làm việc ở cấp nền tảng với ranh giới quyền khác người dùng tenant. Thiết bị cần một danh tính ổn định, credential MQTT và tập topic được phép sử dụng.



Hình 2.1. Ca sử dụng trọng tâm của hệ thống

Nguồn: Nhóm tác giả xây dựng từ mã nguồn.

Bảng 2.1. Yêu cầu chức năng trọng tâm

Mã	Yêu cầu	Kết quả nghiệp vụ
FR01	Xác thực và phân quyền người dùng	Xác định tenant, role và permission cho request
FR02	Quản lý danh mục Device	Tạo, xem, sửa, tìm kiếm và lọc theo UID/trạng thái
FR03	Tổ chức theo project và group	Gán thiết bị có sẵn, xem theo workspace
FR04	Provisioning thiết bị	Cấp thông tin kết nối gắn với UID
FR05	Quản lý datastream	Định nghĩa pin, alias, kiểu và chiều dữ liệu
FR06	Nhận telemetry, status và heartbeat	Lưu dữ liệu và cập nhật last_seen
FR07	Xác định online/offline	Tính trạng thái hiệu lực theo bản tin và timeout

FR08	Điều khiển thiết bị	Phát command đúng UID và theo dõi ACK
FR09	Trình bày trên web/mobile	Hiển thị danh sách, chi tiết và lịch sử

Bảng 2.1 hệ thống hóa yêu cầu chức năng trọng tâm. Hình 2.1 thể hiện ca sử dụng trọng tâm của hệ thống. Các yêu cầu không tách rời nhau. FR02 tạo Device và UID để FR04 cấp cấu hình. FR05 tạo ngữ nghĩa cho kênh mà FR06 nhận. FR07 dùng các mốc thời gian do FR06 cập nhật. FR08 dựa vào cùng UID và quyền truy cập của FR01. Giao diện ở FR09 chỉ có thể hiển thị đúng khi toàn bộ đường dữ liệu phía sau giữ được mối quan hệ này.

Bảng 2.2. Yêu cầu phi chức năng và ảnh hưởng thiết kế

Nhóm	Yêu cầu	Ảnh hưởng
Định danh	UID duy nhất và an toàn cho topic	Ràng buộc unique; grammar không chứa /,+,#
Cách ly	Dữ liệu không chéo tenant hoặc device	Guard API, tenant context, RLS và ACL
Khả dụng	Một payload lỗi không làm dừng subscriber	Validate theo bản tin và xử lý lỗi cục bộ
Quan sát	Có last_seen và nguyên nhân trạng thái	Lưu mốc thời gian, status, heartbeat
Nhất quán	HTTP success không thay ACK thiết bị	Theo dõi command_id và event phản hồi
Mở rộng	Truy vấn lịch sử có giới hạn	Index theo device/time, limit và khoảng thời gian
Bảo mật	Không lộ token/credential trong UI hoặc báo cáo	Hash/rotate/revoke và ẩn dữ liệu nhạy cảm

Kiến trúc phân lớp

Bảng 2.2 hệ thống hóa yêu cầu phi chức năng và ảnh hưởng thiết kế. Backend được tổ chức theo modular monolith và bounded context. Presentation nhận request HTTP hoặc sự kiện giao tiếp, chuẩn hóa dữ liệu vào và trả response. Application chứa luồng sử dụng và các port. Infrastructure hiện thực repository, SQLAlchemy session, MQTT subscriber/publisher và tích hợp lưu trữ. Domain giữ thực thể và quy tắc cốt lõi. Router API v1 ghép các context vào một ứng dụng FastAPI, giúp giao diện có một điểm truy cập thống nhất nhưng vẫn giữ ranh giới mã nguồn.

Ở phía thiết bị, firmware và simulator không gọi trực tiếp repository. Chúng chỉ giao tiếp qua hợp đồng MQTT. Ở phía người dùng, web/mobile không truy cập broker hoặc database mà gọi API. Sự phân tách này làm rõ trách nhiệm: broker định tuyến, backend kiểm tra và diễn giải, database lưu trạng thái, giao diện trình bày. Khi một lỗi xảy ra, có thể xác định nó thuộc kết nối, parser, nghiệp vụ, lưu trữ hay hiển thị thay vì coi toàn bộ nền tảng là một khối.

Bảng 2.3. Trách nhiệm của các lớp và thành phần

Thành phần	Đầu vào	Trách nhiệm	Đầu ra
Web/Android	Thao tác người dùng	Hiển thị, validate giao diện, gọi API	HTTP request, SSE subscription
FastAPI presentation	HTTP request/JWT	Xác thực, schema, permission	Use case hoặc HTTP response
Application/domain	Dữ liệu đã kiểm tra	Quy tắc Device, project, command	Lệnh repository/publisher
MQTT subscriber	Topic và payload	Parse UID, validate, cập nhật/lưu	Device status, Telemetry, ACK
Repository/DB	Thao tác dữ liệu	Transaction, tenant context, quan hệ	Bản ghi nhất quán
ESP32/simulator	Config MQTT, command	Đọc/gửi dữ liệu, phản hồi	Telemetry, status, heartbeat, events

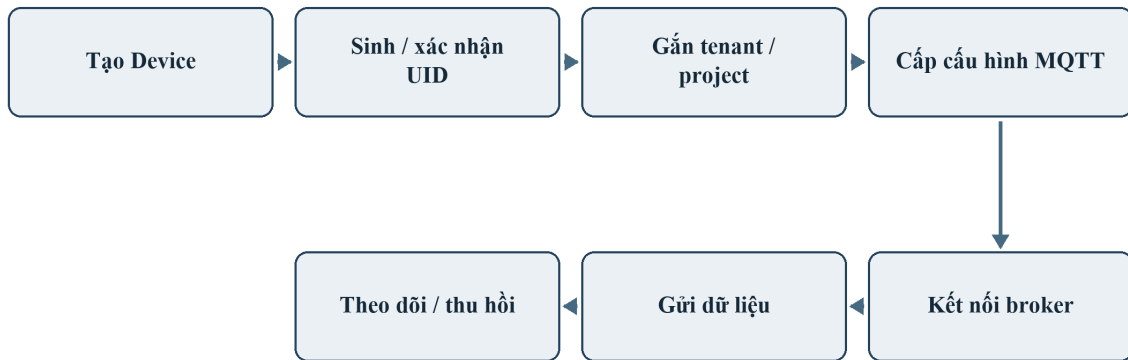
2.2. Thiết kế quản lý thiết bị và dữ liệu

Biểu diễn Device và vai trò của UID

Bảng 2.3 hệ thống hóa trách nhiệm của các lớp và thành phần. Device là điểm neo của toàn bộ nghiệp vụ. Bản ghi có id dạng UUID dùng cho quan hệ nội bộ và device_uid duy nhất dùng trong giao tiếp. Ngoài tên và metadata, Device có tenant_id, project_id tùy chọn, trạng thái, last_seen_at, offline_timeout_seconds và các trường liên quan credential. Việc có cả id và UID giúp database dùng khóa ổn định trong khi firmware sử dụng chuỗi có thể cấu hình. UID phải duy nhất trên phạm vi mà backend tra cứu; nếu hai bản ghi dùng cùng UID, subscriber không thể quyết định thiết bị nhận dữ liệu.

UID còn tham gia vào topic MQTT, đường dẫn API và thao tác giao diện. Chuỗi cho phép chỉ gồm ký tự an toàn, tránh dấu phân cấp và wildcard. Đây không phải giới hạn hình thức: dấu “/” sẽ tạo thêm một mức topic, còn “+” hoặc “#” mang nghĩa wildcard. Nếu UID không được kiểm soát, một thiết bị có thể làm sai parser hoặc mở rộng phạm vi subscription ngoài dự kiến.

Vòng đời đăng ký và provisioning



Hình 2.2. Vòng đời của một thiết bị trong nền tảng

Nguồn: Nhóm tác giả xây dựng từ mã nguồn.

Hình 2.2 thể hiện vòng đời của một thiết bị trong nền tảng. Vòng đời bắt đầu khi người dùng hoặc luồng provisioning tạo Device. Hệ thống xác nhận UID chưa tồn tại và gán phạm vi sở hữu. Project có thể chưa được chọn ở bước đầu; thiết bị chưa gán vẫn xuất hiện tại trang toàn tenant. Sau khi có bản ghi, API cung cấp host, port, username hoặc token và các topic cần thiết. Secret gốc không được lưu ở dạng có thể đọc lại trong giao diện; các luồng rotate/revoke dùng để thay đổi hoặc vô hiệu hóa credential.

Sau kết nối, status hoặc heartbeat cập nhật last_seen_at. Người dùng có thể đổi tên, metadata, timeout hoặc gán project mà không đổi UID dùng bởi firmware. Khi chuyển thiết bị giữa project, dữ liệu lịch sử vẫn thuộc Device, còn datastream và dashboard phải được diễn giải theo project hiện hành. Khi thu hồi, credential cần bị vô hiệu hóa trước khi bản ghi được xóa hoặc ngừng sử dụng để client cũ không tiếp tục publish.

Bảng 2.4. Trạng thái vòng đời và thao tác quản lý tương ứng

Trạng thái nghiệp vụ	Thông tin tối thiểu	Thao tác phù hợp
Chưa phân bổ	UID, tên, tenant có thể rộng theo luồng admin	Kiểm tra inventory, phân bổ tenant
Đã thuộc tenant	UID, tenant, credential/provisioning	Gán project hoặc chuẩn bị kết nối
Đã gán project	Project và datastream liên quan	Hiển thị trong workspace, cấu hình kênh
Đang kết nối	Status/heartbeat, last_seen	Theo dõi telemetry, gửi command được phép
Mất kết nối	Last_seen vượt timeout	Cảnh báo vận hành, không giả định lệnh đã tới
Thu hồi	Credential revoked, trạng thái quản trị	Ngăn kết nối mới, lưu dấu vết cần thiết

Bảng 2.4 hệ thống hóa trạng thái vòng đời và thao tác quản lý tương ứng. Bảng 2.5 là mô hình nghiệp vụ rút ra từ các trường và API hiện có, không phải một enum duy nhất trong database. Mục đích là giúp người dùng hiểu Device đang ở đâu trong vòng đời. Một thiết bị chưa có telemetry không đồng nghĩa bản ghi lỗi; nó có thể mới được tạo hoặc chưa được cấp nguồn. Ngược lại, một Device có lịch sử dữ liệu nhưng credential đã thu hồi cần được giữ ở trạng thái quản trị phù hợp thay vì xóa mọi dấu vết.

Quản lý credential và thay đổi quyền sở hữu

Credential MQTT liên kết với UID nhưng không nên được coi là thuộc tính bất biến suốt đời. Khi nghi ngờ lộ thông tin, quản trị viên cần rotate credential và ngăn credential cũ kết nối. Khi Device chuyển tenant, chỉ đổi tenant_id là chưa đủ: ACL, mapping và token liên quan phải được cập nhật theo cùng một transaction nghiệp vụ hoặc một quy trình có trạng thái trung gian rõ. Nếu không, client cũ có thể tiếp tục gửi vào namespace hợp lệ nhưng dữ liệu được diễn giải dưới chủ sở hữu mới.

Mô hình hiện có các trường hash và route rotate/revoke, tạo cơ sở cho vòng đời này. Tuy vậy, báo cáo chưa có phép thử runtime về việc credential cũ bị broker từ chối sau rotate. Đây là ví dụ điển hình của khác biệt giữa cấu trúc đã triển khai và hành vi đã xác minh. Khi bổ sung test, cần dùng broker test, ghi thời điểm rotate, thử cả credential mới và cũ, sau đó xác nhận không có telemetry ngoài dự kiến.

Tenant, project và device group

Tenant là ranh giới sở hữu. Project là không gian tổ chức công việc trong tenant, không phải một bản sao của tenant. Một Device có thể thuộc tenant nhưng chưa có project. Trang ClientDevices phục vụ góc nhìn toàn tenant, vì vậy phải hiển thị cả nhóm đã gán và chưa gán. WorkspaceDevices nhận project hiện tại và chỉ hiển thị các Device thuộc phạm vi đó. Thiết kế này giải quyết tình huống thiết bị vừa được nhập kho hoặc mới onboarding nhưng chưa quyết định nơi sử dụng.

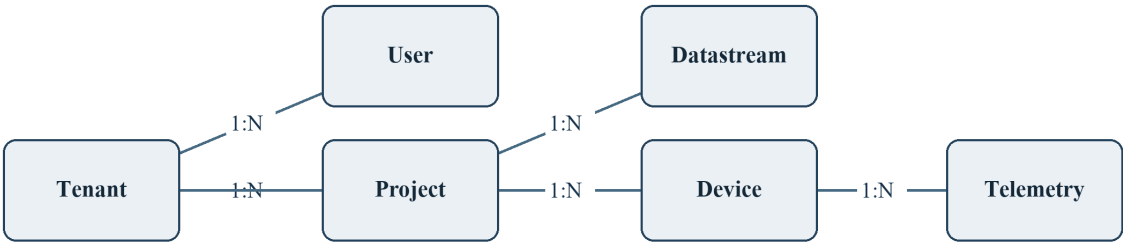
Device group tạo một tập hợp phục vụ lọc hoặc thao tác nhiều đích. Membership không thay tenant_id hay project_id của Device. Khi phát lệnh theo nhóm, hệ thống phải tạo kết quả cho từng mục tiêu; một Device thất bại không được che bởi trạng thái thành công của các Device còn lại. Vì vậy command dispatch và target được tách: dispatch mô tả một lần thao tác, target giữ trạng thái theo từng Device.

Datastream, virtual pin và telemetry

Datastream mô tả kênh dữ liệu trong project bằng pin 0–255, alias, data_type và direction. Ví dụ minh họa, project “Khu A” có V0 là nhiệt độ và V2 là relay. Đây là cấu hình ngữ nghĩa, không phải số đo. Telemetry là các bản ghi phát sinh: Device A có thể gửi 25,5 tại V0 ở một thời điểm, còn Device B gửi giá trị khác tại cùng V0. Cả hai dùng cấu hình kênh giống nhau nhưng vẫn tách bằng device_id.

Virtual pin xuất hiện trong hậu tố ch_vN của topic. Với chiều thiết bị lên, subscriber ánh xạ channel sang metric hoặc datastream tương ứng. Với chiều xuống, publisher dùng commands/ch_vN để điều khiển một kênh. Ràng buộc

unique(project_id,pin) ngăn hai định nghĩa khác nhau dùng cùng pin trong một project. Hai project vẫn có thể dùng V2 với ý nghĩa khác nhau; bởi vậy mọi ánh xạ phải xét cả project lẫn pin.



Device.tenant_id và Device.project_id có thể rỗng

Hình 2.3. Quan hệ dữ liệu phục vụ quản lý thiết bị

Nguồn: Nhóm tác giả xây dựng từ mã nguồn.

Bảng 2.5. Các thực thể dữ liệu chính

Thực thể	Quan hệ quan trọng	Vai trò
Tenant	1–N User, Project, Device	Phạm vi sở hữu và cách ly
User	N–1 Tenant; role/permission	Danh tính người vận hành
Project	N–1 Tenant; 1–N Device/Datastream	Không gian tổ chức nghiệp vụ
Device	UID unique; tenant_id; project_id nullable	Đối tượng kết nối và nguồn dữ liệu
Telemetry	N–1 Device; timestamp; metric	Bản ghi đo theo thời gian
TenantDatastream	N–1 Project; unique(project,pin)	Định nghĩa kênh/virtual pin
DeviceGroup/Member	N–N Device	Tập hợp thiết bị
CommandDispatch/Target	1–N mục tiêu Device	Theo dõi lệnh theo từng đích

ERD rút gọn ở Hình 2.3 ưu tiên những quan hệ trả lời câu hỏi dữ liệu thuộc về ai và thuộc thiết bị nào. Device.tenant_id cùng tenant context tại transaction giữ ranh giới tổ chức. Device.project_id cho phép lọc workspace nhưng nullable để hỗ trợ trạng thái chưa gán. Telemetry.device_id là khóa quan trọng nhất của lịch sử đo. Datastream thuộc project thay vì từng Device để nhiều thiết bị cùng một dự án dùng chung định nghĩa kênh.

Cách tách dữ liệu giữa hai thiết bị minh họa

Giả sử A và B là hai UID minh họa trong cùng tenant. Cả hai được gán vào hai project khác nhau và cùng gửi một metric tên temperature. Topic của A được parser ánh xạ sang Device A, từ đó bản ghi nhận device_id của A. Topic B thực hiện chuỗi tương tự. Khi người dùng mở workspace của project A, API trước hết giới hạn tập

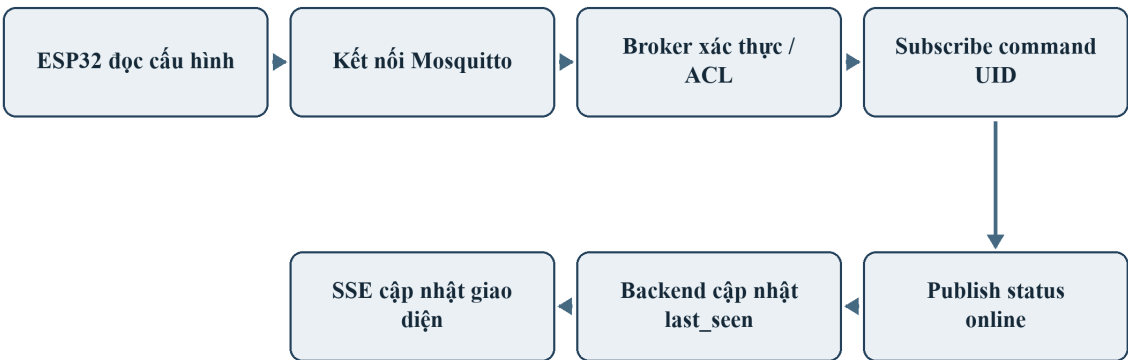
Device theo tenant và project, sau đó chỉ lấy telemetry của A. Tên metric giống nhau không làm hai chuỗi nhập lại vì khóa truy vấn không chỉ dựa trên metric_name.

Nếu A chưa gán project, nó vẫn xuất hiện trong danh sách tenant nhưng không nằm trong workspace nào. Trạng thái này hữu ích cho quy trình onboarding: nhóm có thể kiểm tra kết nối trước rồi mới gán nơi sử dụng. Khi A được gán vào project, lịch sử Device không bị tạo lại. Datastream của project mới quyết định cách dữ liệu tiếp theo được trình bày; dữ liệu cũ cần được truy vấn theo Device và thời gian để tránh bị hiểu nhầm là dữ liệu của cấu hình kênh mới.

2.3. Thiết kế giao tiếp, giám sát và điều khiển thiết bị

Luồng 1 - Thiết bị kết nối

Firmware nhận cấu hình Wi-Fi và MQTT, tạo client_id/username theo thông tin provisioning rồi mở kết nối tới Mosquitto. Broker kiểm tra credential và ACL. Sau sự kiện kết nối, client đăng ký topic commands hoặc commands/ch_vN của chính UID và publish status online. Subscriber backend đã đăng ký wildcard sẽ nhận status, tách UID và cập nhật Device. Người dùng nhìn thấy trạng thái mới qua lần tải REST tiếp theo hoặc sự kiện SSE.



Hình 2.4. Trình tự kết nối và thiết lập hiện diện

Nguồn: Nhóm tác giả xây dựng từ mã nguồn.

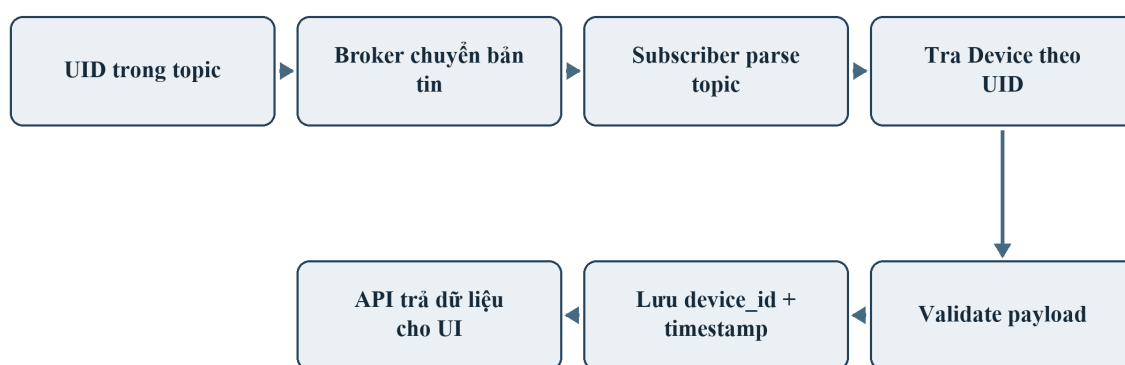
Hình 2.4 thể hiện trình tự kết nối và thiết lập hiện diện. Đầu vào của luồng là một Device đã tồn tại và credential hợp lệ. Kết quả mong muốn không chỉ là socket MQTT được mở mà còn là UID được nhận diện và trạng thái hiển thị đúng. Nếu broker từ chối credential, Device không nên được chuyển online. Nếu status có topic sai, subscriber cần loại bỏ riêng bản tin đó thay vì dừng tiến trình nhận dữ liệu của các UID khác.

Luồng 2 - Thiết bị gửi telemetry

Bảng 2.6. Topic MQTT trong phạm vi báo cáo

Topic	Chiều	QoS thiết kế	Nội dung
devices/{uid}/telemetry	Thiết bị → backend	0	Một hoặc nhiều metric

devices/{uid}/telemetry/ch_vN	Thiết bị → backend	0	Giá trị virtual channel
devices/{uid}/status	Thiết bị → backend	1	Online/offline và metadata trạng thái
devices/{uid}/heartbeat	Thiết bị → backend	Theo hợp đồng client	Dấu hiệu còn kết nối
devices/{uid}/events	Thiết bị → backend	1	ACK hoặc kết quả command
devices/{uid}/commands	Backend → thiết bị	1	Lệnh có command_id và params
devices/{uid}/commands/ch_vN	Backend → thiết bị	1	Giá trị điều khiển virtual pin



Hình 2.5. Luồng telemetry giữ nguyên định danh thiết bị

Nguồn: Nhóm tác giả xây dựng từ mã nguồn.

Hình 2.5 thể hiện luồng telemetry giữ nguyên định danh thiết bị. Bảng 2.6 hệ thống hóa topic mqtt trong phạm vi báo cáo. Subscriber nhận topic và payload độc lập. Topic quyết định UID; payload mang metric, schema version, timestamp hoặc giá trị channel. Sau khi tra được Device, handler kiểm tra kiểu dữ liệu và tách từng metric. Bản ghi lưu device_id, timestamp, metric_name, giá trị số nếu có và raw_payload cho dữ liệu gốc. Với payload chứa nhiều metric, mỗi phần cần giữ cùng Device nhưng có thể tạo các projection riêng phục vụ truy vấn.

Timestamp do thiết bị gửi có thể khác thời điểm backend nhận. Khi đồng hồ client sai, thứ tự biểu đồ có thể bị ảnh hưởng. Thiết kế cần quy tắc cho timestamp vắng, quá xa tương lai hoặc quá cũ. Hệ thống hiện lưu timestamp và dữ liệu gốc, nhờ đó có thể phân tích lại; báo cáo chưa có phép thử runtime đủ để kết luận mọi trường hợp lệch đồng hồ đã được xử lý.

Luồng 3 - Xác định online và offline

Một bản tin hợp lệ cho thấy thiết bị đã xuất hiện tại thời điểm nhận, nên last_seen_at được cập nhật. Presence monitor so khoảng cách thời gian với offline_timeout_seconds. Nếu thiết bị gửi thông điệp offline chủ động, trạng thái có thể đổi ngay; nếu mất nguồn đột ngột, timeout tạo quyết định thay thế. Ngưỡng cho

phép cấu hình trong khoảng 10–600 giây để cân bằng giữa phát hiện nhanh và mạng không ổn định.

SSE gửi thay đổi trạng thái từ server tới trình duyệt. REST vẫn cần cho snapshot ban đầu và phục hồi khi stream đứt. Giao diện không nên giữ nhãn online vô thời hạn chỉ vì sự kiện offline bị mất; khi reconnect, nó cần làm mới snapshot. Trạng thái của từng Device được tính từ last_seen riêng, vì vậy A hết timeout không được kéo B sang offline.

Ảnh hưởng của QoS, retained message và reconnect

QoS 0 phù hợp với telemetry có chu kỳ vì mất một mẫu không nhất thiết làm mất ý nghĩa chuỗi, trong khi việc lưu lặp do retry có thể tăng dữ liệu. QoS 1 phù hợp hơn cho command và status nhưng có khả năng giao trùng; handler vì vậy không nên giả định mỗi bản tin chỉ đến một lần. command_id hỗ trợ nhận diện ACK lặp, còn timestamp và khóa nghiệp vụ cần được xem xét cho telemetry nếu client retry. Việc chọn QoS là cân bằng giữa chi phí, độ tin cậy và khả năng xử lý trùng, không phải cam kết tuyệt đối rằng bản tin sẽ được thiết bị thực thi.

Retained status có thể giúp subscriber mới biết trạng thái gần nhất, nhưng cũng có nguy cơ phát lại một giá trị cũ sau reconnect. last_seen và timestamp phải được dùng để tránh retained message cũ ghi đè quan sát mới. Khi thiết bị reconnect, firmware đăng ký lại topic lệnh và publish trạng thái hiện tại. Backend cần xử lý sự kiện theo UID mà không tạo một Device mới. Các trường hợp retained cũ, reconnect nhanh và ACK lặp nên nằm trong bộ test tích hợp vì chúng khó đánh giá chỉ qua đường đi thành công.

Luồng 4 và 5 - Gửi command và nhận phản hồi



Hình 2.6. Chuỗi điều khiển và xác nhận lệnh

Nguồn: Nhóm tác giả xây dựng từ mã nguồn.

Hình 2.6 thể hiện chuỗi điều khiển và xác nhận lệnh. Yêu cầu lệnh chứa mục tiêu, loại command và params. Backend xác nhận Device thuộc phạm vi của người dùng, tạo command_id rồi publish tới topic của UID. Firmware chỉ xử lý tập command mà handler hỗ trợ. Sau xử lý, thiết bị gửi event chứa command_id và kết quả; backend dùng khóa này để cập nhật target. Nếu không có ACK, trạng thái phải

giữ pending hoặc chuyển failed theo chính sách, không tự gán success từ kết quả publish.

Virtual pin là biến thể của cùng luồng. Giao diện gửi giá trị cho V2, publisher phát commands/ch_v2 và thiết bị phản hồi telemetry/ch_v2 hoặc event. UID vẫn là lớp định tuyến thứ nhất, pin là lớp định tuyến thứ hai. Vì cùng V2 có thể xuất hiện ở nhiều project, backend phải xác định Device và project trước khi diễn giải ý nghĩa của pin.

Xử lý dữ liệu sai và bảo vệ giao tiếp

Parser từ chối topic sai số đoạn, UID rỗng, ký tự wildcard hoặc channel ngoài phạm vi. Validator từ chối JSON không hợp lệ, schema không hỗ trợ và kiểu metric không phù hợp. Lỗi của một bản tin được ghi nhận có kiểm soát để worker tiếp tục xử lý bản tin tiếp theo. Giới hạn kích thước payload và tần suất cũng cần được xem xét khi triển khai thực tế, nhưng chưa có số đo tải để đặt một con số trong báo cáo.

Ở broker, ACL hạn chế namespace theo UID; ở API, JWT và permission bảo vệ thao tác người dùng; ở database, tenant context và RLS bổ sung cách ly hàng. Ba lớp giải quyết ba kiểu truy cập khác nhau. Credential thiết bị cần có vòng đời rotate/revoke, không dùng chung cho cả fleet. Bản in và ảnh chụp phải che password, token, cookie và chuỗi kết nối.

2.4. Thiết kế giao diện và luồng sử dụng hệ thống

Mục tiêu giao diện

Giao diện cần giúp người vận hành trả lời nhanh ba câu hỏi: có những thiết bị nào, thiết bị nào cần chú ý và thao tác tiếp theo là gì. Danh sách vì vậy kết hợp tên, UID, project, trạng thái và last seen thay vì chỉ hiển thị một thẻ tên. Bộ lọc all/online/offline và tìm kiếm giảm thời gian dò trong fleet. Các thao tác tạo, sửa, xóa hoặc điều khiển phải xuất hiện theo permission để viewer không được mời thực hiện hành động mà API sẽ từ chối.

Trang danh sách và onboarding

ClientDevices là góc nhìn toàn tenant. Người dùng có thể tìm theo tên hoặc UID, lọc trạng thái, mở detail và nhận biết thiết bị chưa gán project. WorkspaceDevices dùng cùng dữ liệu cơ sở nhưng đặt trong project; tại đây người dùng gán Device có sẵn hoặc tạo mới. Việc tách hai trang tránh tạo bản ghi trùng khi một thiết bị chỉ cần chuyển ngữ cảnh quản lý.

Onboarding hướng người dùng qua các bước chọn loại thiết bị, nhập thông tin, tạo bản ghi và nhận cấu hình kết nối. Màn hình cần nhấn mạnh UID và credential chỉ xuất hiện ở thời điểm phù hợp. Sau khi lưu, người dùng quay về danh sách để quan sát kết nối thay vì tự gán định thiết bị đã online. Nếu UID trùng hoặc thiếu quyền, lỗi cần xuất hiện tại bước tạo, không để đến lúc broker kết nối mới phát hiện.

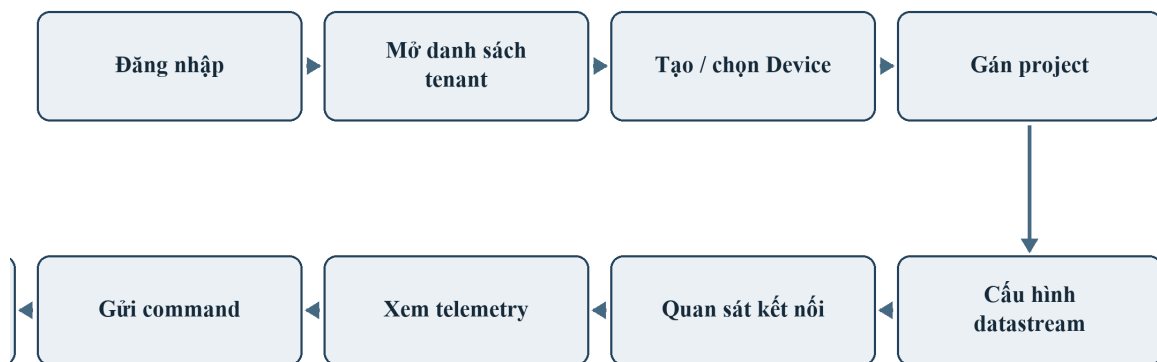
Device detail, datastream và telemetry

Trang chi tiết kết hợp metadata, trạng thái, last_seen và dữ liệu gần nhất. Lịch sử telemetry cần cho phép chọn metric hoặc khoảng thời gian, đồng thời cho biết khi dữ liệu đang tải, không có dữ liệu, hay request bị lỗi. Ba trạng thái này khác nhau về nghiệp vụ. Một biểu đồ rộng do lỗi mạng không được trình bày như cảm biến chưa từng gửi số đo.

WorkspaceDatastreams quản lý alias, pin, kiểu và direction. Người dùng cấu hình kênh trước khi dùng widget hoặc lệnh virtual pin. Khi một pin đã tồn tại trong project, giao diện phải báo trùng thay vì tạo hai ngữ nghĩa cho cùng topic. Mỗi quan hệ giữa datastream và Device cần được giải thích trên UI: datastream thuộc project, dữ liệu thực tế vẫn được tách theo Device.

Command và phản hồi

Màn hình điều khiển cho phép chọn thiết bị hoặc tập mục tiêu, cấu hình tham số và theo dõi trạng thái. Thông tin quan trọng nhất sau thao tác là trạng thái theo target. API accepted, broker published và device acknowledged là ba mốc khác nhau. UI nên hiển thị rõ command_id hoặc thời gian để người dùng liên hệ một phản hồi với lần gửi tương ứng, đặc biệt khi thao tác lặp lại nhanh.



Hình 2.7. Hành trình người dùng quản lý một thiết bị

Nguồn: Nhóm tác giả xây dựng từ mã nguồn.

Hình 2.7 ghép các màn hình thành một hành trình thay vì xem mỗi trang như một CRUD riêng. Người dùng có thể dừng sau bước quan sát nếu thiết bị chưa kết nối. Chỉ khi status, datastream và dữ liệu phù hợp mới nên thực hiện lệnh. Mobile đóng vai trò truy cập nhanh cho danh sách và chi tiết; các chức năng chỉ có trên web cần được ghi rõ để tránh tạo kỳ vọng hai client hoàn toàn tương đương.

Bảng 2.7. Thông tin ưu tiên trên từng màn hình

Màn hình	Thông tin chính	Quyết định người dùng
Danh sách tenant	Tên, UID, project, status, last_seen	Tìm thiết bị cần xử lý hoặc chưa gán
Workspace project	Device thuộc project, nhóm và trạng thái	Gán/bỏ gán, làm việc theo ngữ cảnh

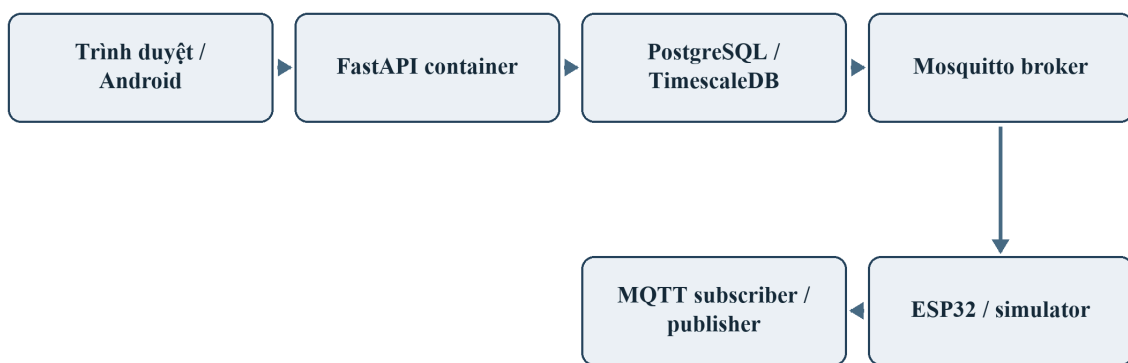
Onboarding	Loại Device, UID, thông tin kết nối	Tạo bản ghi và cấu hình client
Device detail	Metadata, last_seen, telemetry gần nhất/lịch sử	Đánh giá kết nối và dữ liệu
Datastream	Pin, alias, kiểu, direction	Chuẩn hóa kênh cho project
Command	Mục tiêu, params, command_id, target status	Theo dõi gửi và ACK theo thiết bị
Mobile detail	Thông tin tóm tắt phù hợp màn hình nhỏ	Quan sát nhanh, thao tác được hỗ trợ

Bảng 2.7 hệ thống hóa thông tin ưu tiên trên từng màn hình. Thiết kế responsive cần ưu tiên khả năng nhận biết hơn mật độ. Trên màn hình nhỏ, tên và trạng thái có thể đặt trước metadata phụ; UID cần sao chép được nhưng không chiếm toàn bộ chiều ngang. Bảng web có chức năng thay đổi độ rộng cột và lưu lựa chọn, được kiểm tra bằng component test. Với thao tác nguy hiểm như xóa hoặc thu hồi credential, hộp thoại xác nhận phải nêu đúng thiết bị, tránh người dùng nhầm giữa hai dòng có tên gần giống nhau.

CHƯƠNG 3. TRIỂN KHAI, THỬ NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG

3.1. Môi trường và quá trình triển khai

Môi trường dự án gồm frontend Vite/React, backend FastAPI, PostgreSQL/TimescaleDB, Mosquitto, ứng dụng Android, firmware ESP-IDF và simulator Python. Docker Compose định nghĩa các service backend, database, broker và thành phần hỗ trợ trên cùng network. Frontend gọi API qua cổng công bố; ESP32 hoặc simulator kết nối tới cổng MQTT. Android sử dụng API HTTP tương tự web ở các luồng đã hiện thực.



Hình 3.1. Bố trí các thành phần trong môi trường triển khai

Nguồn: Nhóm tác giả xây dựng từ mã nguồn.

Bảng 3.1. Thành phần triển khai và vai trò vận hành

Thành phần	Cách chạy trong dự án	Kết nối chính
Frontend	Vite trên host hoặc cấu hình web	HTTP API, SSE
Backend	FastAPI trong container API	PostgreSQL, Mosquitto, HTTP
Database	PostgreSQL/TimescaleDB container	SQLAlchemy session
Broker	Mosquitto container	MQTT client thiết bị và backend
ESP32	ESP-IDF firmware	Wi-Fi và MQTT
Simulator	Python client/fleet simulator	MQTT hoặc luồng mô phỏng tương ứng
Android	Ứng dụng Java	HTTP API
Kiểm thử web	Vitest	Component với context/API giả lập

Bảng 3.1 hệ thống hóa thành phần triển khai và vai trò vận hành. Hình 3.1 thể hiện bố trí các thành phần trong môi trường triển khai. Quá trình khởi động phụ thuộc biến môi trường, port và thứ tự sẵn sàng của database/broker. Makefile và

script Windows cung cấp lệnh khởi động, seed và mô phỏng. Seed hoặc test ghi dữ liệu không được dùng chỉ để lấy ảnh nếu chưa xác nhận database đích. Một lần demo có thể tái lập cần ghi commit, thời điểm, container đang chạy và UID được sử dụng nhưng không đưa secret vào biên bản.

Tại thời điểm kiểm tra ngày 27/09/2026, API trả OpenAPI trên cổng 8002; PostgreSQL và Mosquitto đang có container tương ứng. Không có frontend container đang chạy, vì vậy báo cáo không tạo screenshot UI. Các vị trí ảnh ở mục 3.2 được giữ để nhóm chụp từ một phiên đăng nhập thật và thay vào trước khi nộp.

Trình tự triển khai cho một phiên demo có thể tái lập

Một phiên demo nên bắt đầu bằng kiểm tra cấu hình chỉ đọc: tên database, host, port API, port broker và commit hiện tại. Tiếp theo khởi động stack, kiểm tra health của backend và kết nối broker. Chỉ sau khi target database được xác nhận là môi trường demo/test mới chạy seed cần thiết. Hai Device thử nghiệm được tạo với UID để phân biệt nhưng không dùng secret thật. Simulator hoặc ESP32 kết nối lần lượt, mỗi client phát một metric khác nhau để quan sát sai lệch ánh xạ nếu có.

Trong lúc demo, nhóm lưu ba nhóm bằng chứng có cùng timestamp: log MQTT cho biết topic/UID; response API hoặc dòng DB cho biết device_id; ảnh giao diện cho biết người dùng nhìn thấy gì. Sau đó gửi command tới một UID, lưu command_id và ACK. Cuối cùng ngắt riêng một client quá timeout rồi ghi trạng thái của cả hai. Quy trình này bao phủ danh mục, dữ liệu, hiện diện và điều khiển mà không cần benchmark lớn. Nếu một bước thất bại, log được giữ lại như kết quả thay vì bỏ qua và chỉ chụp màn hình cuối.

3.2. Kết quả xây dựng hệ thống

Danh sách nhiều thiết bị và tổ chức theo project

Sản phẩm có hai góc nhìn danh sách. Trang tenant tổng hợp Device, đếm trạng thái và cho phép tìm kiếm/lọc. Trang workspace thu hẹp theo project và hỗ trợ gán thiết bị có sẵn. Cách bố trí này phù hợp với vòng đời đã thiết kế: Device có thể tồn tại trước project, sau đó được phân bổ mà không đổi UID. Ý nghĩa đối với người dùng là có thể quản lý cả kho thiết bị lẫn phạm vi triển khai cụ thể.

VỊ TRÍ BỔ SUNG ẢNH THỰC TẾ

Chụp trang ClientDevices sau khi đăng nhập: thấy tối thiểu hai UID, cột trạng thái, project, last seen và bộ lọc. Che token, email nhạy cảm và credential.

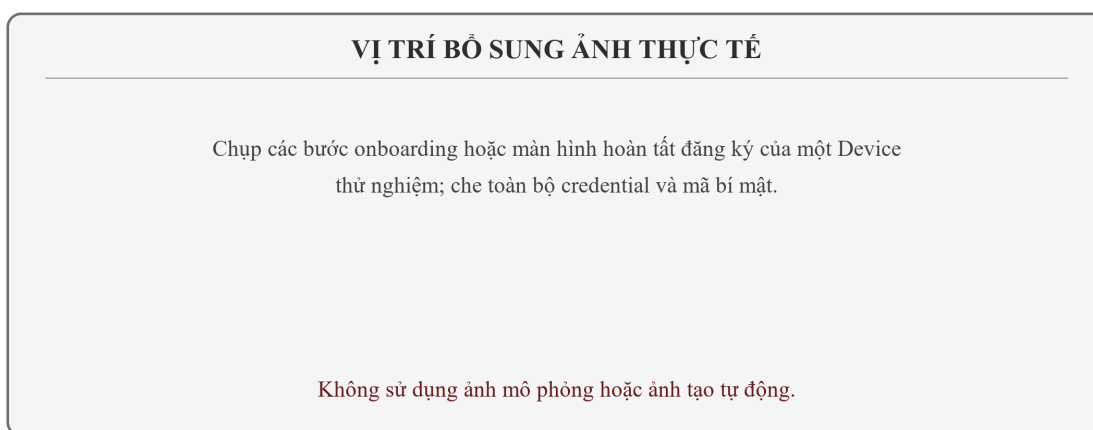
Không sử dụng ảnh mô phỏng hoặc ảnh tạo tự động.

Hình 3.2. Giao diện danh sách nhiều thiết bị

Nguồn: Nhóm tác giả bổ sung từ hệ thống đang chạy.

Tạo thiết bị và cấp cấu hình kết nối

Hình 3.2 được giữ chỗ cho giao diện danh sách nhiều thiết bị; bằng chứng chỉ có giá trị sau khi thay bằng ảnh runtime. Luồng onboarding tạo Device và trả thông tin cần thiết cho client kết nối. UID được kiểm tra trước khi lưu; tenant lấy từ phiên người dùng thay vì request body tùy ý. Sau khi tạo, thiết bị xuất hiện trong danh sách và có thể gán project. Phần kết quả cần thể hiện rõ màn hình hoàn tất hoặc chi tiết Device, nhưng tuyệt đối không để password hoặc token thật trong ảnh.



Hình 3.3. Luồng tạo và đăng ký thiết bị

Nguồn: Nhóm tác giả bổ sung từ hệ thống đang chạy.

Datastream, Device Detail và telemetry

Hình 3.3 được giữ chỗ cho luồng tạo và đăng ký thiết bị; bằng chứng chỉ có giá trị sau khi thay bằng ảnh runtime. WorkspaceDatastreams cho phép định nghĩa virtual pin theo project. Device detail dùng UID để tải metadata, trạng thái và telemetry. Khi dữ liệu có nhiều metric, giao diện chọn đúng chuỗi và hiển thị thời gian. Mức sản phẩm ở đây gồm cấu hình kênh và màn hình đọc; việc chứng minh một giá trị vừa được ESP32 gửi cần ảnh giao diện đi kèm log MQTT hoặc truy vấn DB cùng thời điểm.



Hình 3.4. Giao diện datastream và dữ liệu thiết bị

Nguồn: Nhóm tác giả bổ sung từ hệ thống đang chạy.

Trạng thái kết nối và điều khiển

Hình 3.4 được giữ chỗ cho giao diện datastream và dữ liệu thiết bị; bằng chứng chỉ có giá trị sau khi thay bằng ảnh runtime. Subscriber xử lý status, heartbeat và telemetry để cập nhật last_seen; SSE cung cấp thay đổi tới web. Command center và publisher tạo lệnh theo UID; event ACK cập nhật trạng thái mục tiêu. Mã nguồn đã có đủ thành phần của chuỗi này, nhưng kết quả trên phần cứng chưa được xác minh trong phiên lập báo cáo. Ảnh điều khiển chỉ có giá trị khi hiển thị trạng thái chờ/xác nhận và có log phản hồi tương ứng.

VỊ TRÍ BỔ SUNG ẢNH THỰC TẾ

Chụp trạng thái online/offline của hai thiết bị và một command có kết quả thực tế; kèm command_id hoặc thời gian để đối chiếu log MQTT.

Không sử dụng ảnh mô phỏng hoặc ảnh tạo tự động.

Hình 3.5. Giao diện trạng thái và điều khiển thiết bị

Nguồn: Nhóm tác giả bổ sung từ hệ thống đang chạy.

Web và mobile

Hình 3.5 được giữ chỗ cho giao diện trạng thái và điều khiển thiết bị; bằng chứng chỉ có giá trị sau khi thay bằng ảnh runtime. Web là giao diện quản lý đầy đủ hơn với danh sách, workspace, datastream và command. Android có DevicesFragment, DeviceDetailFragment cùng ViewModel liên quan, cho phép trình bày danh sách và chi tiết ở thiết bị di động. Báo cáo chỉ xem mobile là phần kết quả trong phạm vi màn hình thực sự có mã và dữ liệu; không coi ứng dụng tương đương web ở những chức năng chưa được đấu nối hoặc chưa có bằng chứng runtime.

Bảng 3.2. Kết quả xây dựng nhìn từ hành trình người sử dụng

Bước sử dụng	Thành phần đã có	Giá trị mang lại	Bằng chứng còn cần
Đăng nhập và vào tenant	Auth context, JWT/cookie, permission	Thiết lập phạm vi người dùng trước thao tác	Ảnh phiên đăng nhập đã che thông tin
Mở danh sách	ClientDevices và API danh sách	Quan sát fleet, tìm/lọc, nhận biết chưa gán	Hình 3.2 từ runtime
Onboarding Device	Luồng tạo/đăng ký và provisioning	Đưa đầu cuối mới vào danh mục có UID	Hình 3.3 và response đã che secret

Gán project	WorkspaceDevices và project access	Đặt thiết bị vào đúng ngữ cảnh vận hành	Ảnh trước/sau gán trong cùng tenant
Tạo datastream	WorkspaceDatastreams, pin/alias/direction	Chuẩn hóa kênh dùng bởi dashboard và command	Hình 3.4
Theo dõi thiết bị	Detail, telemetry API, status/SSE	Xem dữ liệu và tình trạng theo UID	Ảnh + log/DB cùng timestamp
Điều khiển	Command center, publisher, target/ACK	Gửi lệnh và theo dõi từng đích	Hình 3.5 + ACK thực
Xem trên mobile	DevicesFragment, DeviceDetailFragment	Truy cập nhanh từ điện thoại	Ảnh runtime nếu nhóm sử dụng

Bảng 3.2 không dùng sự tồn tại của một màn hình để thay cho kết quả vận hành. Cột “thành phần đã có” mô tả sản phẩm nhóm đã xây dựng; cột cuối nêu bằng chứng cần bổ sung để người chấm nhìn thấy đúng chức năng. Cách trình bày theo hành trình cũng cho thấy một lỗi ở bước trước ảnh hưởng bước sau: Device chưa được tạo đúng UID thì cấu hình MQTT, telemetry và command đều không thể đối chiếu chính xác.

Từ góc nhìn sử dụng, điểm mạnh của sản phẩm là giữ một đối tượng Device xuyên qua nhiều màn hình. Người dùng không phải tạo lại thiết bị khi chuyển từ danh sách tenant sang workspace, cũng không cần nhớ topic để xem dữ liệu. Project và datastream cung cấp ngữ cảnh; UID và device_id giữ danh tính kỹ thuật. Thiết kế này phù hợp với mục tiêu quản lý tập trung hơn một giao diện chỉ hiển thị các bản tin MQTT thô.

Điểm còn yếu là bằng chứng sản phẩm chưa đồng đều. Các route, model và component có thể đối chiếu, nhưng báo cáo cần ảnh của hệ thống đang chạy để chứng minh bố cục và khả năng thao tác thực tế. Vì người dùng đã yêu cầu không chụp UI trong phiên biên soạn, tài liệu giữ placeholder có chỉ dẫn cụ thể. Trước khi nộp, nhóm nên thay placeholder bằng ảnh thật thay vì xóa toàn bộ; nếu một chức năng chưa chạy, giữ chú thích hạn chế và không tạo ảnh mô phỏng.

3.3. Thử nghiệm quản lý nhiều thiết bị IoT

Nguyên tắc và dữ liệu quan sát

Thử nghiệm được giới hạn ở thao tác đọc hoặc bộ test đã chạy an toàn. Không chạy backend suite trên database dev vì một số fixture kết nối PostgreSQL, thực hiện INSERT/DELETE rồi COMMIT. Kết quả dưới đây phân biệt rõ phép chạy terminal, snapshot dữ liệu đang tồn tại và chức năng mới được xác nhận qua cấu trúc. Nhãn “đạt” chỉ dùng khi lệnh đã chạy và output có assertion thực.

Bảng 3.3. Kết quả thử nghiệm và quan sát đã thực hiện

Mã	Mục tiêu	Điều kiện/thao tác	Kỳ vọng	Kết quả thực tế	Bằng chứng	Đánh giá
----	----------	--------------------	---------	-----------------	------------	----------

T01	API đang phục vụ	GET /openapi.json tại cổng 8002	HTTP 200 và có route Device	HTTP 200; có CRUD, telemetry, status stream	OpenAPI runtime 27/09/2026	Đạt phạm vi route
T02	Chặn truy cập ẩn danh	GET /api/v1/client/devices không token	HTTP 401	HTTP 401	Response runtime	Đạt trường hợp ẩn danh
T03	Component danh sách	Chạy 2 file Vitest	Assertion giao diện đạt	2 file, 13 test đạt	Terminal Vitest 26/09/2026	Đạt với mock
T04	Dữ liệu nhiều Device	SELECT đọc chỉ theo Device	Nhiều Device có dữ liệu tách riêng	8 Device; 6 có telemetry	PostgreSQL snapshot	Quan sát, không phải E2E
T05	Trạng thái từng Device	SELECT status/last_seen	Có trạng thái riêng từng dòng	8/8 đang lưu offline	PostgreSQL snapshot	Quan sát tại một thời điểm
T06	Command và ACK đúng UID	Cần broker + client thử nghiệm	ACK gắn command_id/UID	Chưa có phép chạy an toàn	[CHÈN LOG MQTT THỰC TẾ]	Chưa xác minh
T07	Cách ly tenant	Cần hai tài khoản + DB test	Không đọc UID tenant khác	Chưa chạy	[CHÈN KẾT QUẢ TEST TENANT]	Chưa xác minh

Bảng 3.3 hệ thống hóa kết quả thử nghiệm và quan sát đã thực hiện. T01 xác nhận ứng dụng FastAPI đang công bố đúng nhóm route cần cho đề tài; nó không xác nhận mọi route hoạt động với dữ liệu thật. T02 chỉ chứng minh request không có danh tính bị từ chối, chưa chứng minh tenant A không đọc được tài nguyên của B. T03 có assertion thật nhưng các file dùng vi.mock cho API, AuthContext, FeatureContext và stream. Vì vậy 13 test là bằng chứng cho hành vi component như resize, lưu độ rộng cột, thao tác chạm và hiển thị; chúng không được cộng thành kiểm thử MQTT hoặc CRUD backend.

Cách đọc kết quả và giới hạn sai số

Mỗi loại bằng chứng trả lời một câu hỏi khác nhau. OpenAPI cho biết route được ứng dụng đang chạy công bố. HTTP 401 cho biết middleware/guard từ chối request ẩn danh. Vitest cho biết component phản ứng đúng với dữ liệu và context đã dựng. SELECT cho biết trạng thái database tại một thời điểm. Không bằng chứng nào trong số đó tự chứng minh ESP32 đã publish, ACL đã chặn chéo UID hoặc tenant isolation đúng cho mọi endpoint. Bảng kết quả giữ riêng cột “bằng chứng” để tránh gộp các lớp xác minh.

Snapshot database có thể chứa dữ liệu từ nhiều lần chạy trước và không cung cấp điều kiện đầu vào. Vì vậy số 38.856 được dùng để mô tả dữ liệu đang tồn tại,

không phải benchmark. Tương tự, trạng thái offline của tám Device có thể do client không chạy tại thời điểm truy vấn, không phải kết quả một test timeout có kiểm soát. Để chuyển một quan sát thành test, cần biết trạng thái ban đầu, thao tác, thời gian chờ và kết quả mong đợi trước khi thực hiện.

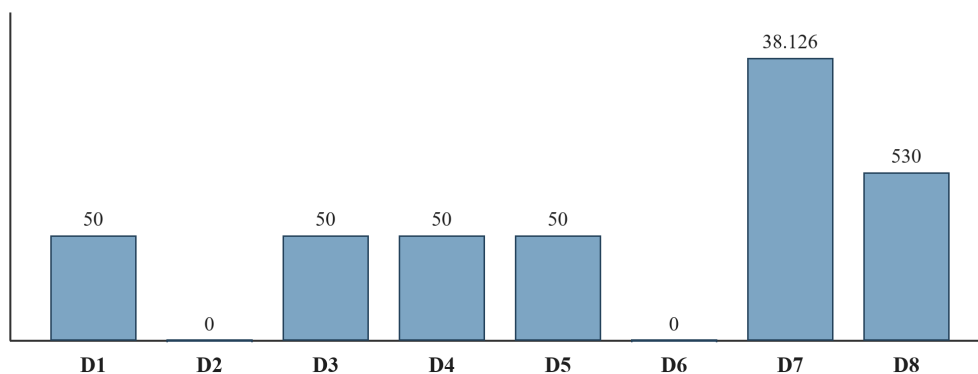
Ảnh chụp dữ liệu nhiều thiết bị

Truy vấn SELECT đọc chỉ tại 00:17:42 ngày 27/09/2026 theo múi giờ Asia/Ho_Chi_Minh ghi nhận 3 tenant, 3 project, 7 datastream, 8 Device và 38.856 dòng Telemetry. Sáu Device có ít nhất một bản ghi. Cả tám dòng Device lưu trạng thái offline tại thời điểm truy vấn. Snapshot chứng minh database có dữ liệu của nhiều Device, nhưng không cho biết các client đã hoạt động đồng thời hay tốc độ ingest.

Bảng 3.4. Số bản ghi theo thiết bị trong snapshot database

Thiết bị ẩn danh	Có tenant	Có project	Telemetry	Bản ghi gần nhất
D1	Có	Có	50	14/09 23:46
D2	Có	Không	0	—
D3	Có	Có	50	14/09 23:46
D4	Có	Không	50	14/09 23:46
D5	Có	Không	50	14/09 23:46
D6	Có	Không	0	—
D7	Có	Có	38.126	16/09 02:29
D8	Không	Không	530	21/09 23:50

Thang logarit được dùng để nhìn thấy cả các thiết bị có 50 bản ghi.



Hình 3.6. Phân bố số bản ghi telemetry trong ảnh chụp dữ liệu ngày 27/09/2026

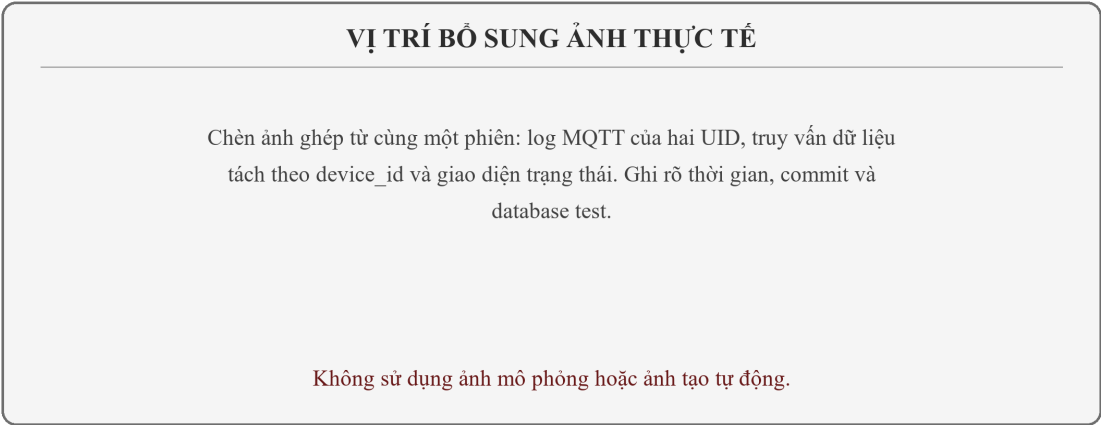
Nguồn: Nhóm tác giả tổng hợp từ truy vấn SELECT đọc chỉ.

Hình 3.6 thể hiện phân bố số bản ghi telemetry trong ảnh chụp dữ liệu ngày 27/09/2026. Bảng 3.4 hệ thống hóa số bản ghi theo thiết bị trong snapshot database. D1–D8 là nhãn ẩn danh sắp theo device_uid, không phải UID mới tạo. Tổng số hàng khớp 38.856. D7 chiếm phần lớn dữ liệu, còn D2 và D6 chưa có telemetry. Phân bố

lệch cho thấy một tổng số duy nhất không đại diện cho mức hoạt động của toàn fleet. D8 không gắn tenant hoặc project, phù hợp khả năng Device tồn tại trước phân bổ nhưng cũng là bản ghi cần được quản trị xem xét trong quy trình thực tế.

Bảng chứng cần bổ sung cho phép thử đầu cuối

Để hoàn tất T06 và T07, nhóm cần tạo database test hoặc schema cô lập, xác nhận connection target rồi dùng hai UID có credential riêng. A và B gửi các giá trị dễ phân biệt; truy vấn theo device_id phải trả đúng chuỗi. Sau đó gửi command tới A, lưu topic, command_id và ACK; B không được nhận lệnh. Khi ngắt A quá timeout, A chuyển offline trong khi B giữ trạng thái. Một tài khoản tenant khác thử mở UID A phải nhận phản hồi không tiết lộ tài nguyên.



Hình 3.7. Bảng chứng phiên thử nghiệm nhiều thiết bị

Nguồn: Nhóm tác giả bổ sung từ hệ thống đang chạy.

Hình 3.7 phải được bổ sung bằng bằng chứng thật; placeholder không được xem là kết quả. Việc dùng cùng một phiên chạy quan trọng vì ảnh UI ở thời điểm khác log có thể không chứng minh quan hệ nhân quả. Mỗi bằng chứng cần che secret nhưng giữ UID thử nghiệm, timestamp và command_id cần thiết cho đối chiếu.

3.4. Đánh giá kết quả

Bảng 3.5. Đối chiếu mục tiêu với mức xác minh hiện tại

Mục tiêu	Kết quả đã xây dựng	Bằng chứng hiện có	Mức đánh giá
Danh mục nhiều thiết bị	UID unique; danh sách tenant/workspace; group	8 Device trong DB; test component	Đã triển khai, một phần runtime
Tổ chức project/datastream	Project nullable; pin unique theo project	3 project, 7 datastream trong snapshot	Có dữ liệu, chưa thử luồng UI
Thu nhận telemetry	Topic theo UID; subscriber; Telemetry	38.856 dòng của 6 Device	Có dữ liệu lưu, nguồn chưa tái lập
Trạng thái online/offline	Status, heartbeat, last_seen, SSE, timeout	Snapshot 8 offline	Có cấu trúc; timing chưa thử

Command/ACK	Publisher, dispatch/target, events	Chưa có log lệnh-ACK cùng phiên	Đã triển khai, chưa runtime verified
Cách ly tenant	JWT/permission, tenant context, RLS, ACL	HTTP 401 cho ẩn danh	Chưa xác minh chéo tenant
Web/mobile	Các màn hình Device và workspace	13 test web với mock; mã Android	Web một phần; mobile chưa runtime

Hệ thống đã xây dựng được một chuỗi quản lý khá đầy đủ ở mức thành phần: Device có UID, có phạm vi tenant/project, có datastream, subscriber nhận dữ liệu, API cung cấp truy vấn và publisher phát lệnh. Snapshot cho thấy database thực tế chứa nhiều Device và telemetry tách theo device_id. OpenAPI và phản hồi 401 xác nhận backend đang phục vụ ở thời điểm kiểm tra. Đây là nền tảng để trình diễn bài toán của đề tài.

Khoảng trống lớn nhất là bằng chứng đầu cuối. Chưa có một phiên được ghi lại từ lúc hai client kết nối, gửi dữ liệu khác nhau, thay đổi trạng thái, nhận lệnh và trả ACK. Cách ly giữa hai tenant và ACL chéo UID cũng chưa được chạy trên môi trường test cô lập. Do đó “implemented” và “runtime verified” được phân biệt tại Bảng 3.5. Sự phân biệt này chỉ cần nêu một lần để tránh biến toàn bộ chương thành báo cáo kiểm toán.

Về khả năng quản lý nhiều thiết bị, thiết kế đã có các điểm cần thiết: định danh duy nhất, topic chứa UID, khóa ngoại Telemetry.device_id, danh sách theo tenant/project và target lệnh theo Device. Tuy nhiên chưa có số đo tải hoặc số client đồng thời. Báo cáo không đưa ra ngưỡng 20, 50 hay 100 thiết bị. Khi có test stack riêng, simulator có thể tạo fleet tăng dần để đo kết nối, ingest, truy vấn và SSE mà không tác động dữ liệu phát triển.

Về trải nghiệm người dùng, mã web hỗ trợ tìm kiếm, lọc, workspace, onboarding và detail. Ảnh thật vẫn cần bổ sung để chứng minh cách các chức năng kết nối trong sản phẩm. Bốn placeholder ở mục 3.2 nêu chính xác nội dung cần chụp. Mỗi ảnh nên dùng cùng tenant và cùng bộ UID thử nghiệm, không lộ credential. Sau khi thay ảnh, caption và danh mục hình phải được cập nhật lại trong Word.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đồ án giải quyết bài toán tập trung hóa việc quản lý nhiều thiết bị IoT. Thay vì theo dõi từng client bằng cấu hình rời, nền tảng dùng Device và UID làm trục xuyên suốt. Tenant và project tổ chức phạm vi; datastream mô tả kênh; telemetry lưu dữ liệu theo device_id; status, heartbeat và last_seen hỗ trợ theo dõi hiện diện; command_id cùng ACK tách việc backend tiếp nhận yêu cầu khỏi kết quả ở thiết bị. Web và mobile cung cấp điểm truy cập cho người vận hành, còn MQTT kết nối ESP32 hoặc simulator với backend.

Kết quả hiện có cho thấy API runtime công bố các route thiết bị, truy cập ẩn danh bị từ chối, database lưu 8 Device và 38.856 dòng telemetry, trong đó 6 Device có dữ liệu. Hai file kiểm thử web đạt 13 assertion trong môi trường mock. Các bằng chứng này xác nhận một phần sản phẩm nhưng chưa đủ kết luận toàn bộ luồng đầu cuối, cách ly tenant, ACL hoặc command ACK đã vận hành đúng. Các vị trí thiếu đã được đánh dấu để nhóm bổ sung từ môi trường test riêng.

Hướng phát triển gần nhất vẫn nằm trong quản lý thiết bị: hoàn thành kịch bản hai UID có log đồng bộ; đo chuyển trạng thái reconnect/offline; hoàn thiện kết quả lệnh theo từng target; quản lý rotate/revoke credential; bổ sung lịch sử thay đổi Device; cải thiện thao tác nhóm và khả năng phục hồi SSE. Sau khi có bằng chứng đúng cho một fleet nhỏ, simulator mới nên được dùng để tăng dần số client và đo giới hạn thực tế. Mọi số liệu mở rộng cần ghi kèm cấu hình, dữ liệu đầu vào và thời điểm chạy.

TÀI LIỆU THAM KHẢO

[1] A. Banks, E. Briggs, K. Borgendale và R. Gupta, biên tập, “MQTT Version 5.0,” OASIS Standard, 07/03/2019. [Trực tuyến]. Có tại: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>. [Truy cập: 27/09/2026].

[2] Espressif Systems, “ESP-MQTT,” ESP-IDF Programming Guide. [Trực tuyến]. Có tại: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/mqtt.html>. [Truy cập: 27/09/2026].

[3] FastAPI, “Tutorial - User Guide.” [Trực tuyến]. Có tại: <https://fastapi.tiangolo.com/tutorial/>. [Truy cập: 27/09/2026].

[4] PostgreSQL Global Development Group, “Row Security Policies,” PostgreSQL Documentation. [Trực tuyến]. Có tại: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>. [Truy cập: 27/09/2026].

[5] Timescale, “Hypertables,” Timescale Documentation. [Trực tuyến]. Có tại: <https://docs.timescale.com/use-timescale/latest/hypertables/>. [Truy cập: 27/09/2026].

[6] Eclipse Foundation, “ACL File Plugin,” Eclipse Mosquitto Documentation. [Trực tuyến]. Có tại: <https://mosquitto.org/documentation/plugins/acl-file/>. [Truy cập: 27/09/2026].

[7] Docker, “How Compose Works,” Docker Documentation. [Trực tuyến]. Có tại: <https://docs.docker.com/compose/intro/compose-application-model/>. [Truy cập: 27/09/2026].

[8] Mozilla, “Using Server-Sent Events,” MDN Web Docs. [Trực tuyến]. Có tại: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events. [Truy cập: 27/09/2026].

[9] Meta Open Source, “Quick Start,” React Documentation. [Trực tuyến]. Có tại: <https://react.dev/learn>. [Truy cập: 27/09/2026].

[10] OpenAPI Initiative, “OpenAPI Specification.” [Trực tuyến]. Có tại: <https://spec.openapis.org/oas/>. [Truy cập: 27/09/2026].

PHỤ LỤC A. MA TRẬN TRUY VẾT CHỨC NĂNG – THÀNH PHẦN TRIỂN KHAI

Bảng A.1. Vết liên kết giữa chức năng và mã nguồn

Chức năng	Giao diện	Backend/DB/MQTT	Tập kiểm thử hoặc hợp đồng
Danh sách và detail	frontend/.../ClientDevices.tsx; WorkspaceDevices.tsx; Android DevicesFragment	device_registry models; tenant router_client.py	ClientDevices.test.tsx; WorkspaceDevices.test.tsx
Đăng ký/provisioning	ClientDeviceOnboarding.tsx	device_registry router.py; tenant router_client.py	Onboarding.test.tsx
Project và group	WorkspaceDevices.tsx	project_dashboard; device_groups	test_tenant_device_race.py
Datastream/virtual pin	WorkspaceDatastreams.tsx	TenantDatastream; project router.py; mqtt_topics.py	test_mqtt_virtual_channels.py
Telemetry	Device detail/dashboard	mqtt_subscriber.py; Telemetry model	mqtt-device-contract.md
Status/heartbeat/SSE	ClientDevices; status hook	mqtt_subscriber.py; status stream route	test_device_status_rule.py; useDeviceStatusStream.test.tsx
Command/ACK	CommandCenter; Android ViewModel	command_center; mqtt_publisher.py; events handler	test_mqtt_protocol_contract.py
Tenant và quyền	Auth/Feature contexts	security.py; db/session.py; RLS migrations	test_tenant_idor_security.py (chỉ chạy DB test cô lập)
ESP32	—	iot/src/Esp32/main/aifom_edge.c	Đối chiếu handler commands và heartbeat
Fleet simulator	—	iot/simulator/simulate_fleet.py	Cần chạy trên môi trường test riêng

PHỤ LỤC B. MẪU BẢN TIN ĐÃ ẪN THÔNG TIN NHẠY CẢM

Topic minh họa: devices/demo-001/telemetry. Payload minh họa: {"schema_version": "1.0", "temperature": 25.5, "humidity": 60.2}. Command minh họa: {"schema_version": "1.0", "command_id": "<uuid>", "command": "set_relay", "params": {"pin": 25, "state": true}}. Các giá trị này

chỉ giải thích định dạng; không phải dữ liệu thử nghiệm hoặc credential của hệ thống.

PHỤ LỤC C. DANH SÁCH BẢNG CHỨNG CẦN BỔ SUNG

Mục	Bảng chứng cần chèn	Điều kiện chấp nhận
Hình 3.2	Danh sách có ít nhất hai UID và trạng thái	Ảnh runtime, che dữ liệu nhạy cảm
Hình 3.3	Các bước onboarding hoặc kết quả tạo Device	Không hiển thị credential thật
Hình 3.4	Datastream và telemetry cùng UID/project	Có timestamp và nguồn Device
Hình 3.5	Command và trạng thái target	Có command_id và ACK/log đối chiếu
Hình 3.7	Log hai UID + DB/API/UI cùng phiên	Ghi commit, thời gian, database test
T06/T07	Output test command/ACK và tenant isolation	Chạy trên database/broker test cô lập